CONFIDENTIAL

CONFIDENTIAL

CONFIDENTIAL

CONFIDENTIAL

İÇİNDEKİLER

Sayfa

1. GİRİŞ 7
1.1 Amaç 7
1.2 Kapsam 7
1.3 Kullanıcı Gizliliği ve Anonimlik 8
2. ANONİM SOSYAL MEDYA PLATFORMLARI 8
3. ÜNİVERSİTE BAZLI ANONİM SOSYAL MEDYA PLATFORMU 10
3.1 Platformun Özellikleri 10
3.1.1 Kullanıcı Yönetimi ve Kayıt 10
3.1.1.1 Kayıt Süreci 10
3.1.1.2 Şifre Yönetimi 11
3.1.1.3 Avatar Özelleştirme 12
3.1.2 Gizlilik ve Görünürlük Kontrolleri 12
3.1.2.1 Profil Gizliliği 12
3.1.2.2 İçerik Görünürlüğü 14
3.1.3 İçerik Oluşturma ve Organizasyonu 15
3.1.3.1 Gönderi Paylaşımı 15
3.1.3.2 Cevap Sistemi 16
3.1.3.3 İçerik Listeleme 16
3.1.4 Etkileşim Mekanizmaları 17
3.1.4.1 Beğeni Sistemi 17
3.1.4.2 Özel Mesajlaşma 17
3.1.5 Bildirim Sistemi 18
3.1.5.1 Bildirim Türleri 18
3.1.5.2 Bildirim Yönetimi 18
3.1.6 Kullanıcı Arayüzü ve Deneyimi 18
3.1.6.1 Tema Seçenekleri 19
3.1.6.2 Erişim Gereklilikleri 20
3.1.7 Destek ve İletişim 20
3.1.8 Moderasyon ve Yönetim 20
3.1.8.1 Moderasyon Araçları 20
3.1.8.2 Yönetim Hiyerarşisi 21
3.2 Temel Kavramlar ve Tanımlar 21
3.2.1 Monolitik Mimari 21
3.2.2 Dağıtık Sistem 22
3.2.3 PWA (Progressive Web App) 23
3.2.4 MVC (Model-View-Controller) 24
3.2.5 Çapraz Kaynak Paylaşımı (CORS) 24
3.2.6 REST API 25
3.2.7 NoSQL Veri Tabanı 25
3.3 Sistem Tasarımı 26
3.4 Kullanıcı Arayüzü (Frontend) 26
3.4.1 Teknoloji Seçimi ve Mimari Yapı 27
3.4.2 Bileşen Organizasyonu 27
3.4.3 Durum Yönetimi ve Veri Akışı 28
3.4.4 Veri İşleme ve Kullanıcı Deneyimi 28
3.4.5 Performans 28

3.4.6 Erişilebilirlik 29
3.5 Sunucu Tarafı (Backend) 29
3.5.1 Proje Yapısı 30
3.5.2 Ara Katman Yazılımları (Middlewares) 31
3.5.2.1 CORS 32
3.5.2.2 Hız Sınırlandırma (Rate Limiting) 32
3.5.2.3 Kimlik Doğrulama ve Yetkilendirme 32
3.5.2.4 Aktivite Takibi 32
3.6 Veri Depolama (Veri Tabanı) 33
3.7 Geliştirme ve Dağıtım Süreci 33
3.7.1 Versiyon Kontrolü 33
3.7.2 Sürekli Entegrasyon ve Sürekli Dağıtım (CI/CD) 33
3.7.3 Dağıtım ve Barındırma 34
3.8 Güvenlik Önlemleri 34
3.8.1 Altyapı Güvenliği 34
3.8.2 Veri Depolama Güvenliği 35
3.8.3 Veri Koruma ve Kimlik Doğrulama 35
3.8.4 İletişim Güvenliği (HTTPS) 35
4. SONUÇLAR VE ÖNERİLER 36
EK A: REST API UÇ NOKTALARI 39
KAYNAKLAR 41
ÖZGEÇMİŞ 42

ŞEKİL LİSTESİ

Sayfa

Şekil 3.1 Kullanıcı Kaydı Sayfası 10
Şekil 3.2 Şifre Değiştirme Sayfası 11
Şekil 3.3 Avatar Özelleştirme 12
Şekil 3.4 Örnek Bir Gizli Profil 13
Şekil 3.5 Örnek Bir Açık Profil 13
Şekil 3.6 Anasayfa Akışı 14
Şekil 3.7 Gönderi Paylaşımı 15
Şekil 3.8 Cevap Yazma 16
Şekil 3.9 Mesaj Kutusu 17
Şekil 3.10 Bildirim Sayfası 18
Şekil 3.11 Koyu Tema 19
Şekil 3.12 Destek Sistemi 20
Şekil 3.13 Admin'in Moderatör Ataması 21
Şekil 3.14 Uygulamanın PWA Olarak Cihaza Yüklenmesi 23
Şekil 3.15 Sistem Tasarımı 26
Şekil 3.17 Backend Dosya Organizasyonu 30

SİMGELER VE KISALTMALAR LİSTESİ

API	: Application Programming Interface
CDN	: Content Delivery Network (İçerik Dağıtım Ağı)
JSON	: JavaScript Object Notation (JavaScript Nesne Notasyonu)
JWT	: JSON Web Token
MongoDB	: Mongo Database (Mongo Veri Tabanı)
REST	: Representational State Transfer (Temsili Durum Transferi)
SEO	: Search Engine Optimization (Arama Motoru Optimizasyonu)

1. GİRİŞ

Üniversite Bazlı Anonim Sosyal Medya Platformu, üniversite öğrencilerine yönelik, tamamen anonim ve metin tabanlı bir sosyal paylaşım platformudur. Platform, kullanıcıların özgürce gönderi paylaşmasına ve cevaplamasına, özel mesajlaşmasına olanak tanır ve mobil cihazlarla uyumlu bir web arayüzü sunar. Paylaşımlar, kullanıcıların bağlı oldukları üniversiteye ait sayfalarda listelenir, böylece yerel düzeyde bir etkileşim ortamı oluşturulur. Platform, mevzuatta zorunlu tutulanlar haricinde hiçbir kullanıcı bilgisini saklamayarak dijital mahremiyete öncelik verir.

1.1 Amaç

Platform, üniversite öğrencilerinin yerel topluluklarına özgü paylaşım yapma konusunda yaşadıkları çekinceleri gidermeyi ve ifade özgürlüklerini artırmayı hedefler. Geleneksel sosyal medya platformlarının geniş kitlelere hitap eden yapısı, üniversite öğrencilerinin gizlilik endişeleri nedeniyle yerel bağlamda paylaşım yapmasını zorlaştırmaktadır. Bu proje, anonim paylaşımı üniversite öğrencilerine odaklayarak mevcut platformların genel kitle odaklı yapısından kaynaklanan bağlam kaybını önlemeyi amaçlar. Böylece, üniversite öğrencileri arasında eleştiri kültürünün gelişmesine ve yerel düzeyde ifade hürriyetinin güçlenmesine katkı sağlanması hedeflenir.

1.2 Kapsam

Üniversite Bazlı Anonim Sosyal Medya Platformu, yalnızca Türkiye Cumhuriyeti sınırları içindeki üniversite öğrencilerine yönelik bir metin tabanlı paylaşım platformu sunar. Proje, görsel paylaşım veya daha geniş bir kullanıcı kitlesini kapsamaz. Platform, kullanıcıların yalnızca benzersiz bir kullanıcı adı ve bağlı oldukları üniversite bilgisiyle kayıt olmasına olanak tanır ve kişisel veri (isim, e-posta adresi,

konum bilgisi vb.) talep etmez. Bu yaklaşım, platformun odaklandığı kullanıcı kitlesine özel bir deneyim sunmayı ve gereksiz veri toplamadan kaçınmayı sağlar.

1.3 Kullanıcı Gizliliği ve Anonimlik

Platform, kullanıcı gizliliğini etik bir sorumluluk olarak ele alır ve bireylerin dijital mahremiyet haklarına saygı gösterir. Kullanıcı adı ve geçmiş paylaşımlar aracılığıyla kimliklerin dolaylı olarak tespit edilebilmesi riskine karşı, isteğe bağlı bir profil gizlilik ayarı sunulur. Bu özellik etkinleştirildiğinde, kullanıcının profil sayfası görüntülenemez ve paylaşımlarında kullanıcı adı gizlenir, böylece içeriklerin anonimliği üst düzeyde korunur. Bu sayede kullanıcılar, kimliklerinin ifşa edilme endişesi olmadan özgürce paylaşım yapabilir ve dijital ortamda güvenli bir etkileşim ortamı sağlanır.

2. ANONİM SOSYAL MEDYA PLATFORMLARI

Anonim sosyal medya platformları, kullanıcıların kimliklerini gizleyerek paylaşım yapmalarına olanak tanıyan sistemlerdir. Bu tür platformlar, özellikle genç kullanıcılar arasında popülerdir ve ifade özgürlüğünü desteklerken, aynı zamanda moderasyon ve güvenlik zorlukları da beraberinde getirmektedir. Yik Yak ve Whisper gibi platformlar, anonim paylaşımın öncüleri olarak bilinmektedir. Yik Yak, özellikle üniversite kampüslerinde yerel topluluklara odaklanarak anonim paylaşım imkanı sunmuş, ancak moderasyon eksiklikleri ve kötüye kullanım nedeniyle 2017 yılında kapanmıştır[2]. Benzer şekilde, Whisper da anonim metin ve görsel paylaşımlara olanak tanımış, ancak kullanıcı verilerinin gizliliği konusunda eleştiriler almıştır . Bu platformlar, anonimliğin getirdiği özgürlüğü sağlarken, içerik denetimi ve kullanıcı güvenliği konularında yetersiz kalmıştır [3].

Türkiye'de anonim sosyal medya platformlarının en bilinen örneklerinden biri Ekşi Sözlük'tür. 1999 yılında Sedat Kapanoğlu tarafından kurulan Ekşi Sözlük, kullanıcıların anonim olarak "entry" adı verilen yorumlarla görüşlerini paylaştığı bir

platformdur. Türkiye'de özellikle üniversite öğrencileri ve genç yetişkinler arasında yaygın bir kullanıma sahip olan Ekşi Sözlük, 2024 yılında 177 milyon tekil ziyaretçi ve 11 milyar sayfa görüntülenmesiyle dikkat çekmektedir[4]. Ancak, platform anonimlikten kaynaklanan sorunlarla da karşılaşmıştır. Örneğin, 2020 yılında pandemi sürecinde uygulanan "65 Yaş Üstü Sokağa Çıkma Yasağı" başlığı altında yapılan paylaşımlarda, yaşlı bireylere yönelik önyargılı ve ayrımcı yorumlar tespit edilmiştir. Bu yorumlarda, yaşlı bireylerin toplum için bir yük olduğu, kurallara uymadığı ve genç nüfusu riske attığı gibi ifadeler kullanılarak nefret söylemi üretilmiştir[5]. Bir diğer örnek olan İnci Sözlük ise daha çok mizahi içeriklere odaklanmakta, ancak sıkça uygunsuz içerik ve erişim engellemeleriyle gündeme gelmektedir. Ekşi Sözlük ve İnci Sözlük gibi platformlar, belirli bir topluluğa odaklanmak yerine, geniş bir kitleye hitap eden ve farklı ilgi alanlarından kullanıcıları bir araya getirerek her türlü konuda paylaşım yapılmasına olanak tanıyan genel bir tartışma alanı sunmayı amaçlamaktadır.

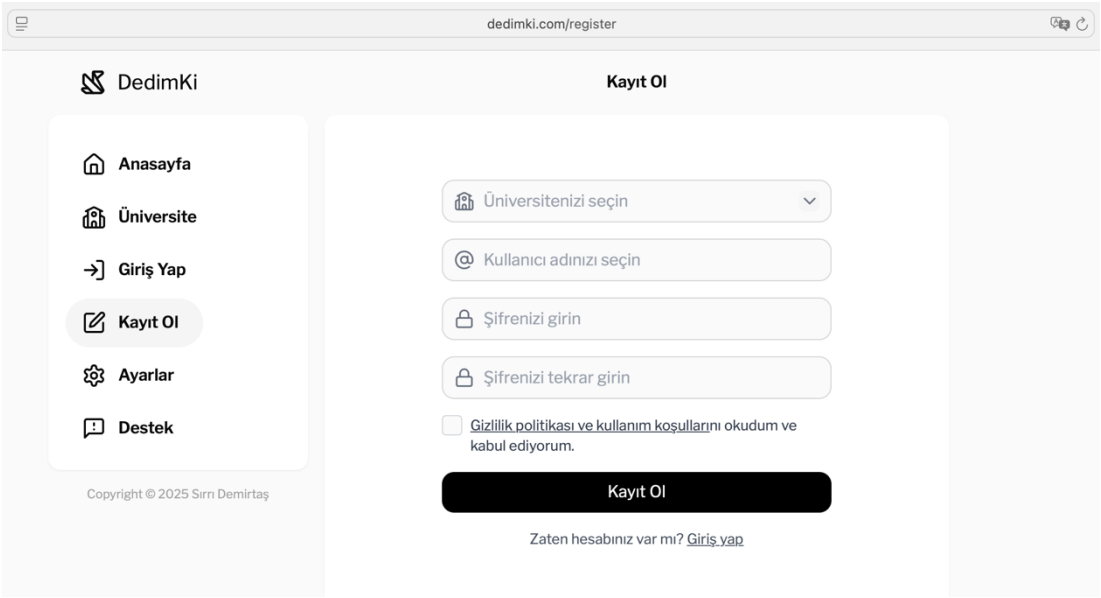
Üniversite Bazlı Anonim Sosyal Medya Platformu, bu çalışmalardan farklı olarak, yalnızca üniversite öğrencilerine yönelik bir yapı sunmakta ve yerel topluluk odaklı iletişimi merkeze almaktadır. Platform, kullanıcıların kendi üniversitelerine ait sayfalarda paylaşım yapmalarını sağlayarak içeriklerin bağlamdan kopmasını engellemekte ve etkili bir moderasyonla içerik denetimini güçlendirmektedir. Bu yönüyle, projenin literatürdeki boşluğu doldurduğu ve anonim paylaşımın getirdiği zorlukları aşmaya yönelik daha kontrollü bir çözüm sunduğu değerlendirilmektedir.

3. ÜNİVERSİTE BAZLI ANONİM SOSYAL MEDYA PLATFORMU

3.1 Platformun Özellikleri

3.1.1 Kullanıcı Yönetimi ve Kayıt

3.1.1.1 Kayıt Süreci



Şekil 3.1 Kullanıcı Kaydı Sayfası

Platform, kullanıcıların yalnızca kullanıcı adı, üniversite bilgisi ve şifreyle kaydolmasını gerektirir. Şekil 3.1'de gösterildiği gibi, bunlar dışında e-posta adresi, cinsiyet, konum gibi kişisel veriler istenmez. Kullanıcı adları:

Minimum 3, maksimum 16 karakter uzunluğundadır.

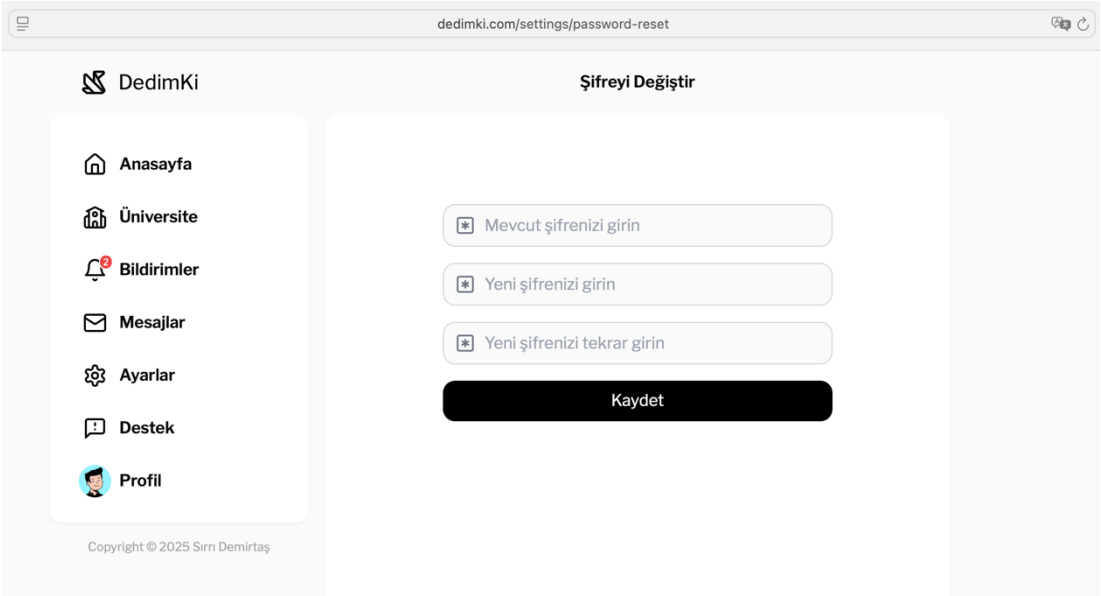
Yalnızca harf ve rakam içerebilir.

Benzersiz olmalıdır.

Kullanıcılar, kayıt oldukları üniversiteyi sonradan değiştiremez.

Gönderi paylaşmak, cevaplamak, beğeni/beğenmeme ifadesi bırakmak, mesaj göndermek için kayıtlı kullanıcı olmak gerekir.

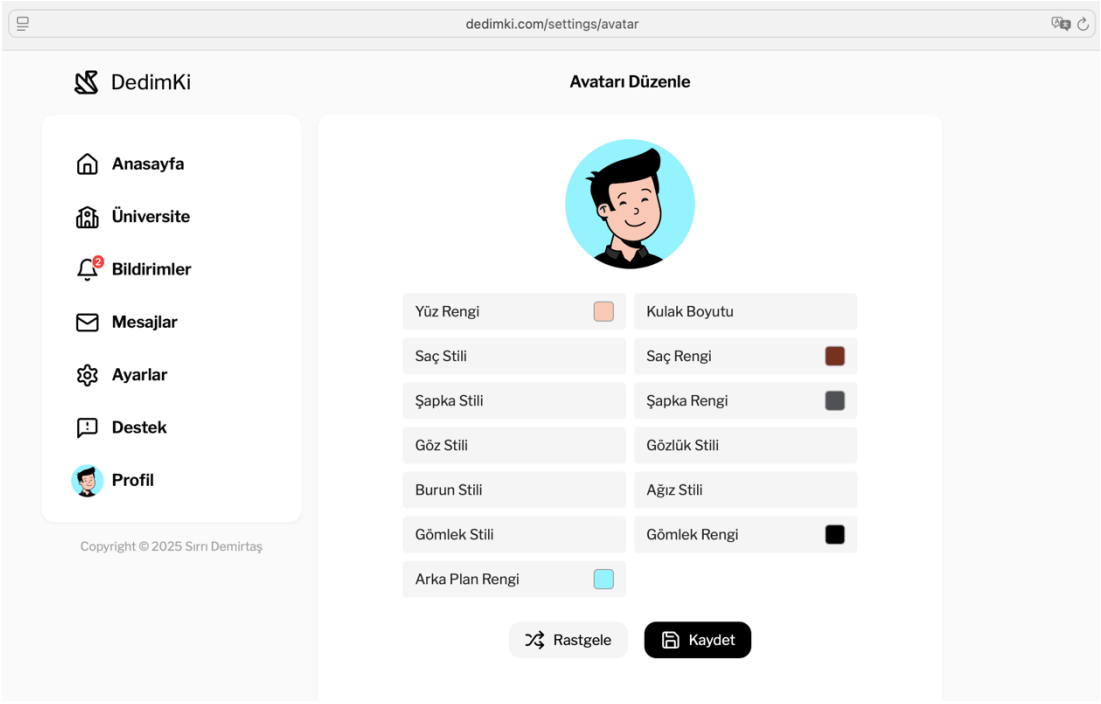
3.1.1.2 Şifre Yönetimi



Şekil 3.2 Şifre Değiştirme Sayfası

Kullanıcılar, Şekil 3.2'de gösterildiği gibi eski şifrelerini kullanarak yeni bir şifre belirleyebilir. Unutulan şifreler için sıfırlama yöntemi yoktur.

3.1.1.3 Avatar Özelleştirme



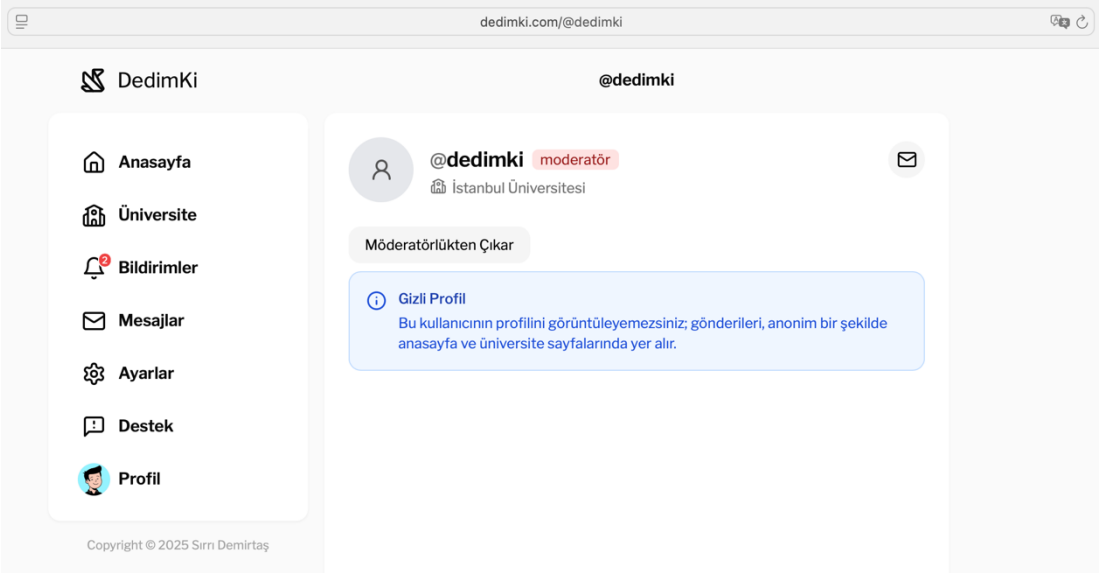
Şekil 3.3 Avatar Özelleştirme

Kullanıcılar, kayıt sonrasında istediği zaman Şekil 3.3'te gösterilen Avatar Düzenleme sayfasında saç rengi, saç şekli, gözlük ve tişört rengi gibi çeşitli seçeneklerle bir avatar oluşturabilir. Bu avatar, gönderilerde ve özel mesajlarda kullanıcı adı ve üniversite bilgisinin yanında görüntülenir, ancak gizli profillere ait paylaşımlarda gizlenir.

3.1.2 Gizlilik ve Görünürlük Kontrolleri

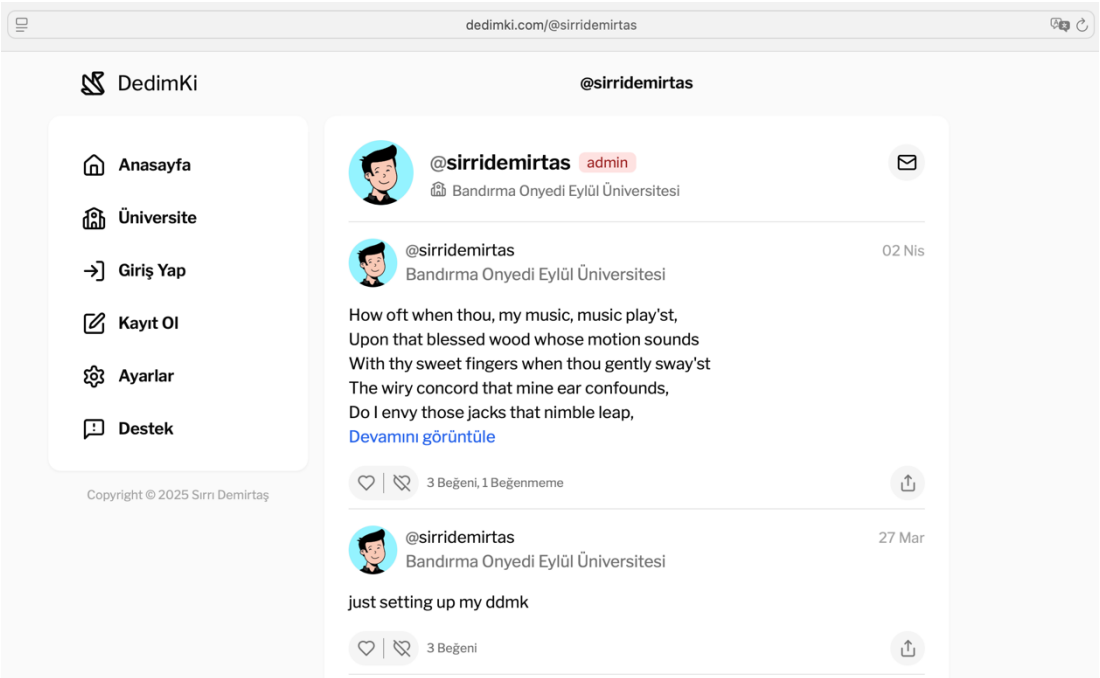
3.1.2.1 Profil Gizliliği

Kullanıcılar, gizlilik ayarlarını "açık" veya "kapalı" olarak belirleyebilir.



Şekil 3.4 Örnek Bir Gizli Profil

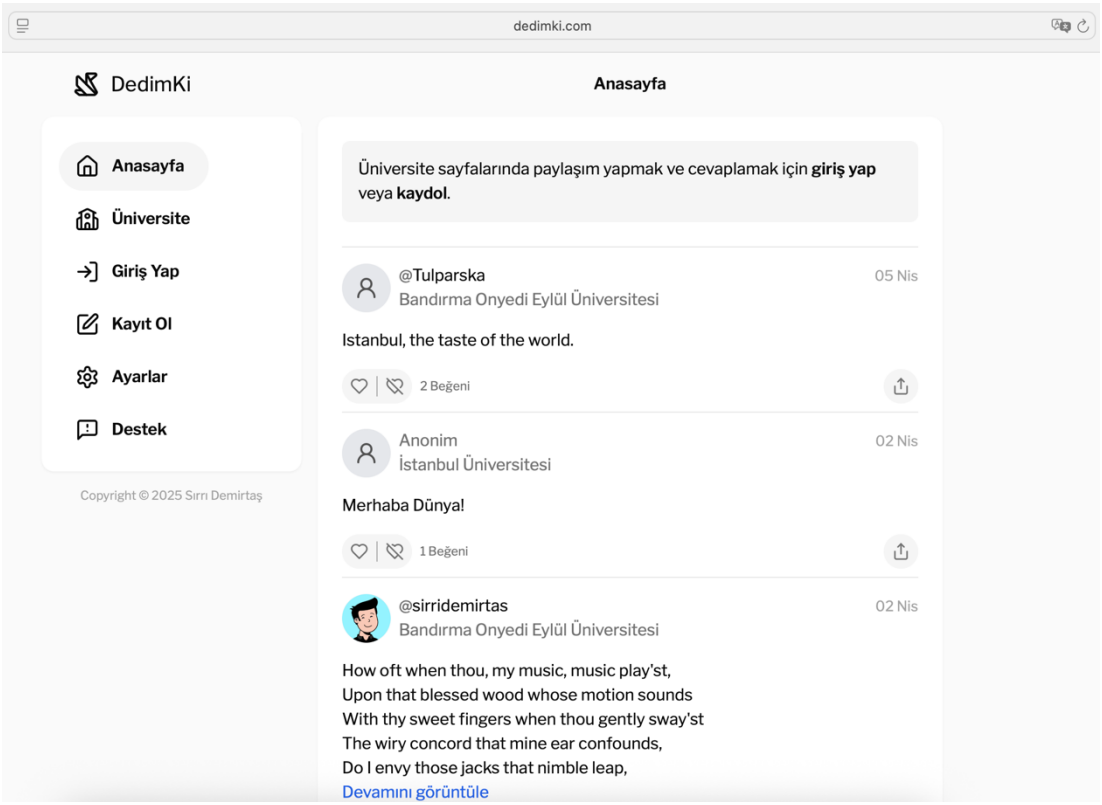
Gizli Profil: Başka kullanıcılar, gizli profillerin gönderilerini yalnızca anasayfa veya üniversite sayfalarında anonim olarak görür; Şekil 3.4'te olduğu gibi kullanıcı adı veya avatar görüntülenmez, üniversite bilgisi görünmeye devam eder.



Şekil 3.5 Örnek Bir Açık Profil

Açık Profil: Şekil 3.5'te olduğu gibi gönderiler, kullanıcı adı ve avatarla birlikte görüntülenir. Gizlilik kaldırıldığında, daha önce anonim olan gönderiler de açık hale gelir.

3.1.2.2 İçerik Görünürlüğü



Şekil 3.6 Anasayfa Akışı

Giriş yapılmış olsun ya da olmasın; tüm gönderiler, anasayfa akışı, üniversite sayfaları, gizli olmayan profiller, yorumlar ve beğeni sayıları kısıtlama olmaksızın görüntülenebilir.

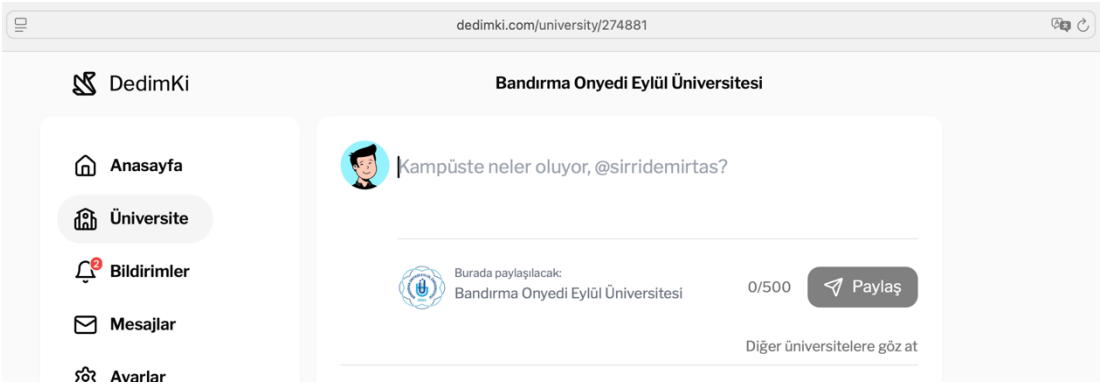
Gönderi veya cevapları beğenen kullanıcılar kimse tarafından görülemez; yalnızca sayıları görünür.

Gönderilerde sistem menüsünden paylaşım ve bağlantı kopyalama için butonlar bulunur.

Şekil 3.6'da gizli bir profilin kullanıcı adının görüntülenemediği ve profiline link olmadığı gösterilmektedir.

3.1.3 İçerik Oluşturma ve Organizasyonu

3.1.3.1 Gönderi Paylaşımı



Şekil 3.7 Gönderi Paylaşımı

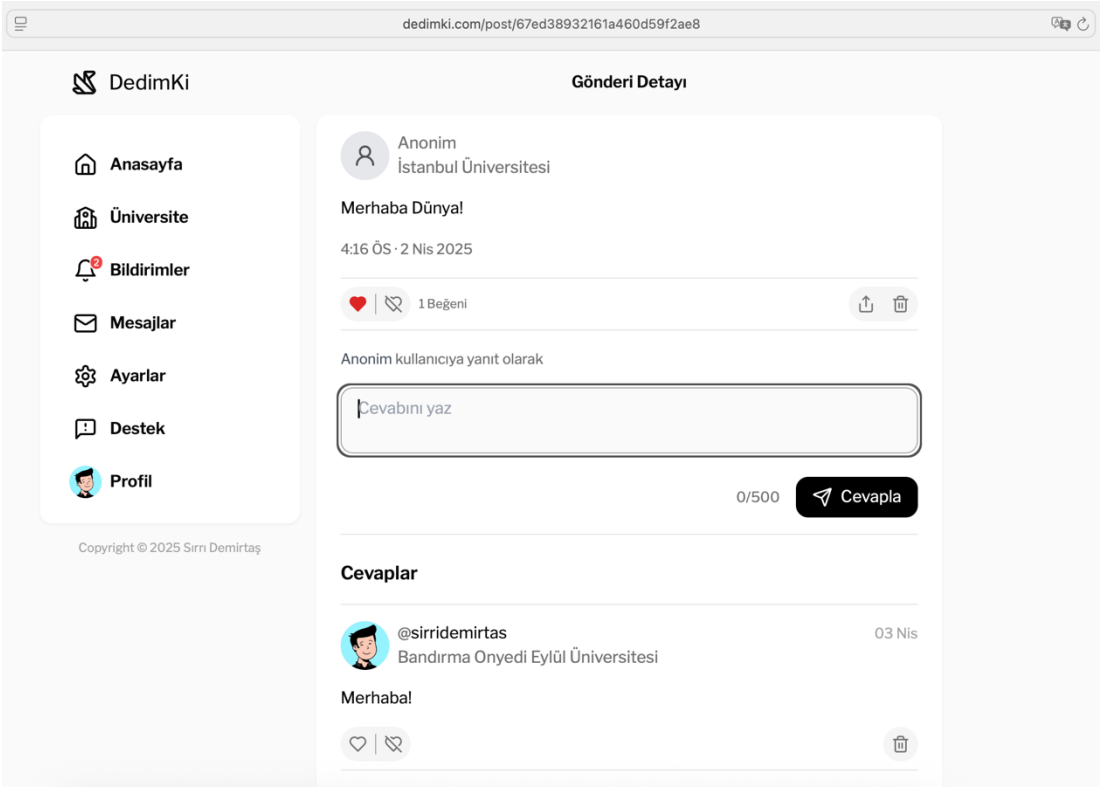
Kullanıcılar Şekil 3.7'de gösterildiği gibi yalnızca metin tabanlı gönderi paylaşabilir. Gönderiler:

Minimum 3, maksimum 500 karakter uzunluğundadır,

Anasayfa (tüm üniversitelerden gönderiler) veya üniversite sayfalarında (yalnızca ilgili üniversiteye ait gönderiler) listelenir.

Kullanıcılar, kendi üniversiteleri dışındaki sayfalarda da gönderi paylaşabilir. Bu durumda gönderinin alt kısmında, hangi üniversitede paylaşıldığını belirten bir ibare yer alır.

3.1.3.2 Cevap Sistemi



Şekil 3.8 Cevap Yazma

Kullanıcılar, Şekil 3.8'de gösterildiği gibi gönderilere minimum 3, maksimum 500 karakterlik cevaplar yazabilir.

Cevaplar, gönderi sayfasında en yenisi en altta olacak şekilde listelenir.

Cevaplara cevap yazılamaz.

3.1.3.3 İçerik Listeleme

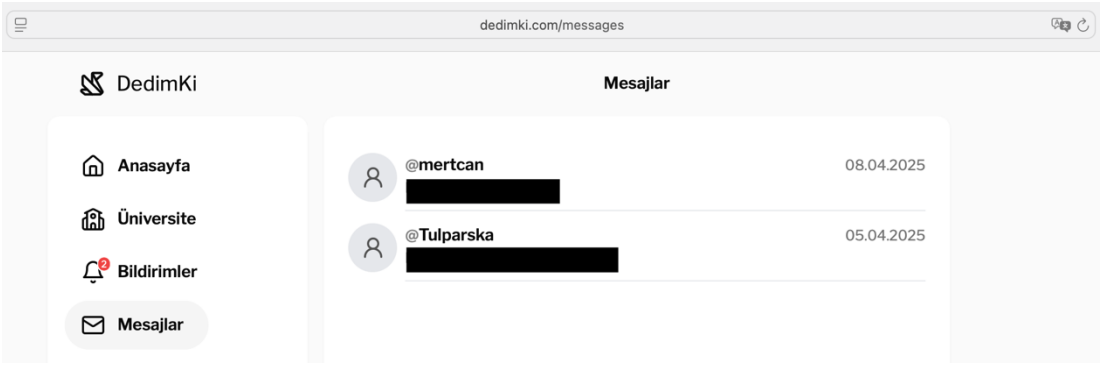
Gönderiler ve cevaplar, "infinity scroll" yöntemiyle sıralanır; aşağı kaydırıldıkça her seferinde 50 içerik daha yüklenir. Bu sistem: anasayfa akışı, üniversite sayfaları, kullanıcı profillerindeki gönderiler, gönderi içindeki cevaplar için geçerlidir. Gönderiler, en yenileri en üstte olacak şekilde düzenlenir.

3.1.4 Etkileşim Mekanizmaları

3.1.4.1 Beğeni Sistemi

Kullanıcılar, gönderilere ve cevaplara beğeni veya beğenmeme ifadesi bırakabilir; bu eylemler geri alınabilir. Beğeni ve beğenmeme sayıları, içeriklerin yanında gösterilir. Beğenen kullanıcıların kimlikleri gizli kalır; yalnızca sayılar görünür.

3.1.4.2 Özel Mesajlaşma

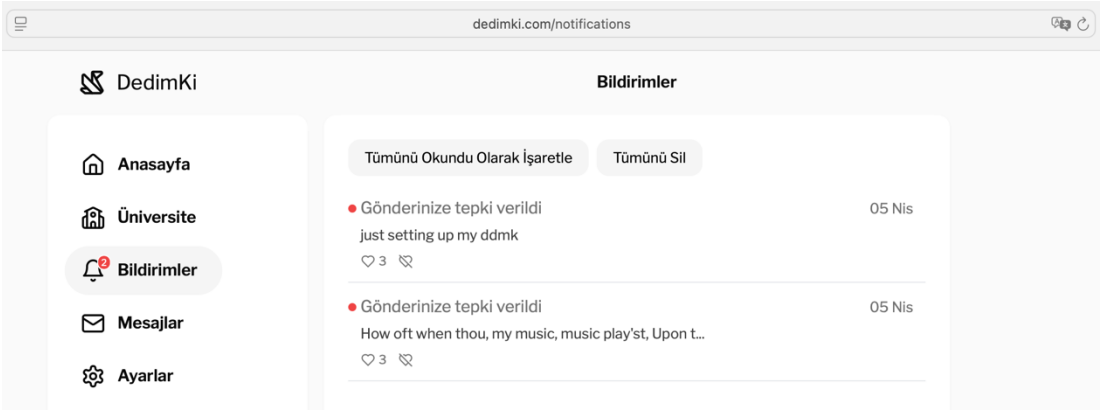


Şekil 3.9 Mesaj Kutusu

Kullanıcılar, birebir özel mesaj gönderebilir. Mesajlar, minimum 3, maksimum 500 karakter uzunluğundadır. Bir konuşmada son 100 mesaj görüntülenir; daha eski mesajlar otomatik silinir. Okunmamış mesaj sayıları, menüdeki mesaj butonunda gösterilir.

Şekil 3.9'da giriş yapmış bir kullanıcının mesaj kutusu gösterilmektedir. İlgili mesaja tıklanarak yazışma ekranına gidilmektedir.

3.1.5 Bildirim Sistemi



Şekil 3.10 Bildirim Sayfası

Kullanıcılar Şekil 3.10'da gösterilen sayfadan bildirimleri takip edebilmekte ve işlem yapabilmektedir.

3.1.5.1 Bildirim Türleri

Kullanıcılar şu durumlarda bildirim alır:

Gönderilerine yorum yapıldığında.

Gönderilerine beğeni veya beğenmeme ifadesi bırakıldığında.

Yorum yaptıkları bir gönderiye başka bir yorum eklendiğinde.

3.1.5.2 Bildirim Yönetimi

Bildirime tıklandığında okunmuş olarak işaretlenir. Tüm bildirimleri okundu olarak işaretleme ve silme seçenekleri vardır. Son 50 bildirim saklanır; daha eskiler otomatik silinir. Okunmamış bildirim sayıları, menüdeki bildirim butonunda görüntülenir.

3.1.6 Kullanıcı Arayüzü ve Deneyimi

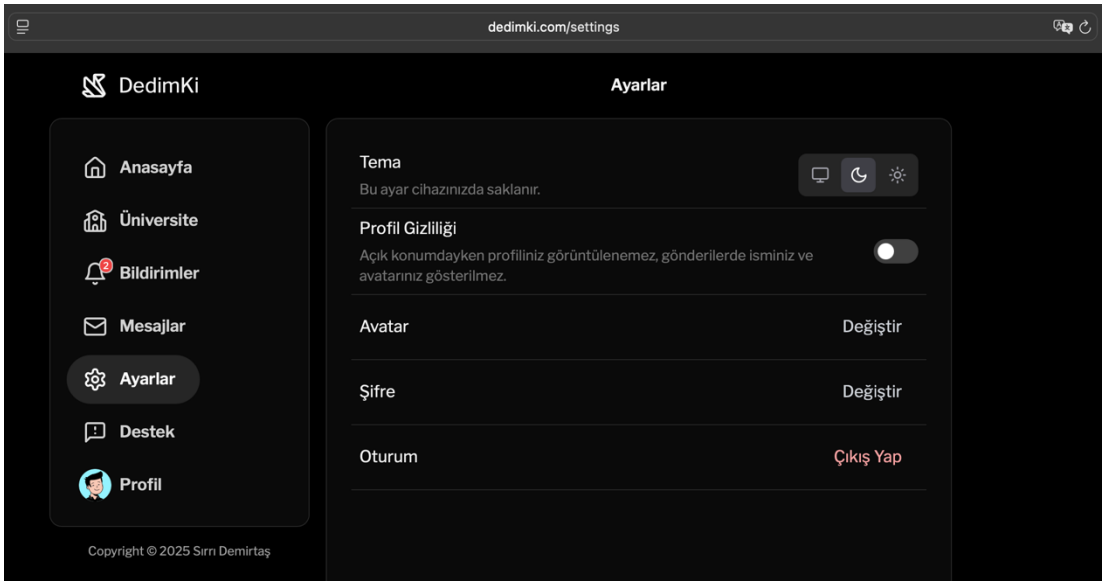
Platformun kullanıcı arayüzü, işlevsellik, estetik ve kullanıcı deneyimini merkeze alan sade ve etkili bir yapıda tasarlanmıştır. X (Twitter), Bluesky ve Threads gibi sosyal medya platformlarının arayüz kalıplarından esinlenilerek; akış temelli gönderi düzeni, sabit gezinme çubuğu ve sade ikonlar gibi tanıdık öğelere yer verilmiştir. Bu sayede kullanıcıların platformu hızlı ve sezgisel biçimde kullanabilmesi hedeflenmiştir.

Tasarımda sadelik, yalnızca görsel bir tercih değil, aynı zamanda kullanıcı deneyimini güçlendiren işlevsel bir strateji olarak benimsenmiştir. Görsel hiyerarşi ve anlamlı bileşen düzenlemeleri ile bilgiye erişimin kolaylaştırılması ve etkileşimin sezgisel hâle getirilmesi amaçlanmıştır.

Tasarım sürecinde Dieter Rams'ın "iyi tasarım" ilkeleri temel anlayış olarak benimsenmiş; özellikle sadelik, açıklık ve işlevsellik ilkeleri doğrultusunda her öğenin anlamlı ve amaca yönelik olması hedeflenmiştir. Renk paleti ve tipografi dikkat dağıtıcı ögelerden kaçınılarak, okunabilirliği ve görsel dengeyi destekleyecek biçimde seçilmiştir. Bu yaklaşım, kullanıcı odaklı ve tutarlı bir arayüz deneyimi sunmayı amaçlamaktadır.

Platformun kullanıcı arayüzü, mobil cihazlarda etkin bir kullanım hedeflenerek responsive tasarım prensipleri çerçevesinde tasarlanmıştır. Bu yaklaşım, farklı ekran boyutları ve çözünürlükler (örneğin, akıllı telefonlar ve tabletler) için arayüzün dinamik bir şekilde uyarlanmasını sağlamış; içerik ve navigasyon öğeleri her cihazda tutarlı bir şekilde sunulmuştur. Responsive tasarım, erişilebilirlik standartlarına uygun olarak geliştirilmiş ve platformun geniş bir kullanıcı kitlesine hitap etmesine katkıda bulunmuştur.

3.1.6.1 Tema Seçenekleri



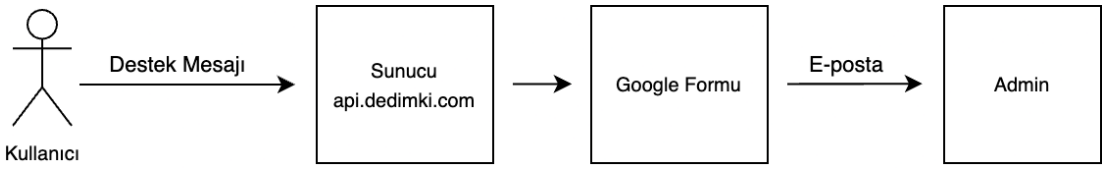
Şekil 3.11 Koyu Tema

Platform, açık ve koyu tema sunar. Varsayılan olarak sistem teması kullanılır; kullanıcıların seçtiği tema tarayıcı önbelleğinde saklanır. Şekil 3.11'de Ayarlar Sayfası koyu temada gösterilmektedir.

3.1.6.2 Erişim Gereklilikleri

Gönderi paylaşmak, cevap yazmak, beğeni/beğenmeme ifadesi bırakmak, bu ifadeleri geri almak ve mesaj göndermek için giriş yapmak gereklidir. Giriş yapılmadan yalnızca görüntüleme işlemleri yapılabilir.

3.1.7 Destek ve İletişim



Şekil 3.12 Destek Sistemi

Kullanıcılar, oturum açma şartı olmadan destek talebi gönderebilir.

Gerekli bilgiler: İsim, e-posta, konu (genel, destek, öneri, teknik, şikayet), mesaj.

Oturum açıkken gönderilen taleplerde yalnızca girilen bilgiler gönderilir. Oturum bilgileri dikkate alınmaz.

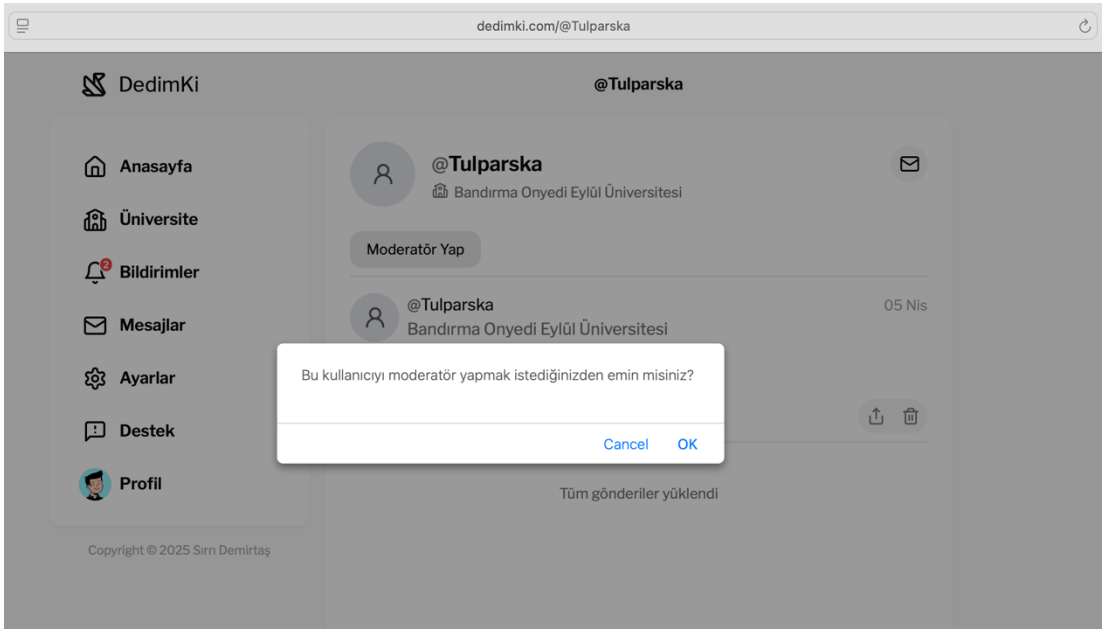
Gönderilen destek mesajları Şekil 3.12'de gösterildiği gibi sunucu tarafında bir Google Form dosyasına kaydedilir ve otomatik gönderilen bir e-posta ile admin kullanıcılara anında bildirilir.

3.1.8 Moderasyon ve Yönetim

3.1.8.1 Moderasyon Araçları

Admin ve moderatörler, tüm gönderileri silme yetkisine sahiptir. İçerikler, hukuki standartlara uygunluk açısından denetlenir.

3.1.8.2 Yönetim Hiyerarşisi



Şekil 3.13 Admin'in Moderatör Ataması

Admin: Platformun tüm yetkilerine sahiptir; moderatör atayabilir veya çıkarabilir.

Moderatör: İçerik kaldırma yetkisine sahiptir. Moderatör atayamaz ya da çıkaramaz.

Şekil 3.13'te admin kullanıcının moderatör ataması gösterilmektedir.

3.2 Temel Kavramlar ve Tanımlar

Bu çalışma kapsamında kullanılan teknik kavramlar, sistem tasarımının ve mimarisinin daha iyi anlaşılabilmesi için aşağıda tanımlanmıştır. Bu kavramlar, hem projenin teknik altyapısını anlamak hem de literatürdeki yerini değerlendirmek açısından önemlidir. Her bir kavram, projenin bağlamında nasıl bir rol oynadığı ile birlikte açıklanmıştır.

3.2.1 Monolitik Mimari

Monolitik mimari, bir yazılım uygulamasının tüm bileşenlerinin (kullanıcı arayüzü, iş mantığı ve veri erişimi) tek bir bütünlük içinde geliştirildiği ve çalıştırıldığı bir tasarım yaklaşımıdır. Bu yapı, uygulamanın tüm işlevselliklerinin tek bir kod tabanında birleştirilmesini sağlar ve genellikle küçük ölçekli projelerde veya geliştirme sürecinin

erken aşamalarında tercih edilir [6]. Monolitik mimarinin avantajları arasında basit bir geliştirme süreci, kolay hata ayıklama ve merkezi bir yönetim yer alırken, ölçeklenebilirlik ve bakım açısından sınırlılıkları bulunmaktadır. Örneğin, uygulamanın bir bileşeninde yapılan bir değişiklik, tüm sistemi etkileyebilir ve bu durum büyük ölçekli sistemlerde karmaşıklığa yol açabilir.

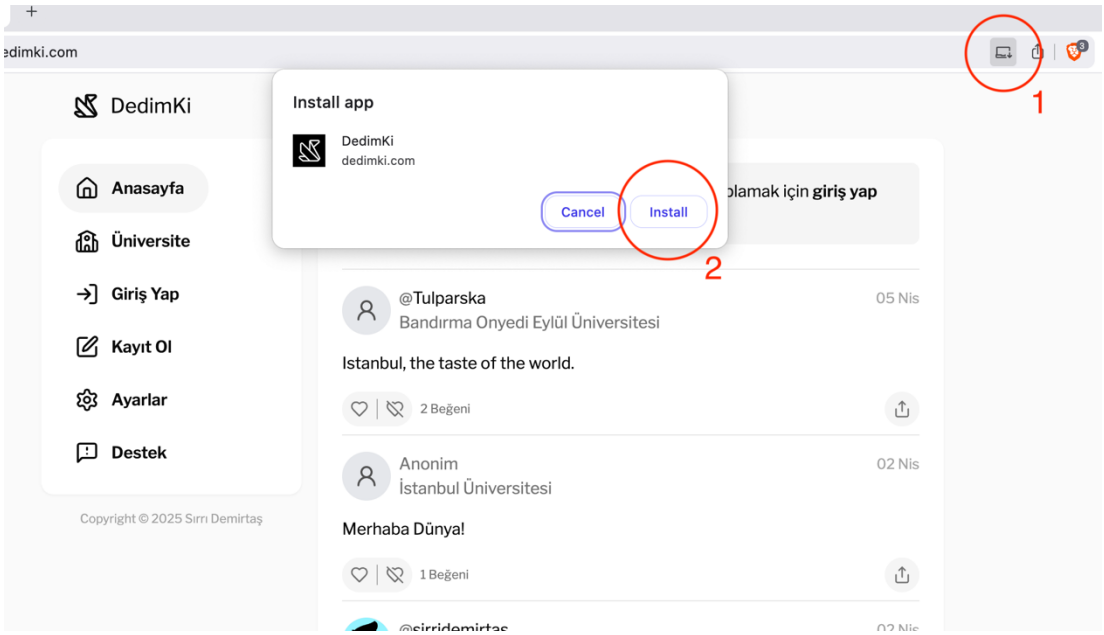
Bu projede, Üniversite Bazlı Anonim Sosyal Medya Platformu başlangıçta monolitik bir mimariyle tasarlanmıştır. Tüm işlevsellikler (kullanıcı arayüzü, iş mantığı ve veri erişimi) tek bir uygulama olarak geliştirilmiş, bu da projenin ilk aşamalarında hızlı bir geliştirme süreci sağlamıştır. Ancak, sistemin kullanıcı arayüzü, sunucu tarafı ve veritabanı farklı platformlarda barındırılmıştır; bu, monolitik yapının dağıtık bir barındırma yaklaşımıyla birleştirildiği hibrit bir tasarım ortaya çıkarmıştır.

3.2.2 Dağıtık Sistem

Dağıtık sistem, birden fazla bileşenin farklı fiziksel veya sanal sunucular üzerinde çalıştığı ve bu bileşenlerin birbiriyle ağ üzerinden iletişim kurduğu bir sistem mimarisidir. Dağıtık sistemler, ölçeklenebilirlik ve yüksek erişilebilirlik gibi avantajlar sunar; çünkü her bileşen bağımsız olarak ölçeklendirilebilir ve bir bileşenin arızalanması durumunda diğerleri çalışmaya devam edebilir [7]. Ancak, bu sistemler, iletişim gecikmeleri, veri tutarlılığı ve koordinasyon zorlukları gibi dezavantajlarla da karşılaşabilir.

Bu projede, kullanıcı arayüzü Cloudflare Pages üzerinde, sunucu tarafı Render platformunda ve veritabanı MongoDB Atlas üzerinde barındırılmıştır. Bu dağıtık barındırma yaklaşımı, her bileşenin bağımsız olarak ölçeklenebilmesini sağlamış ve sistemin performansını artırmıştır. Örneğin, kullanıcı trafiği arttığında yalnızca sunucu tarafı için ek kaynak sağlanarak sistemin kapasitesi artırılabilir. Bu yapı, monolitik mimariyle birlikte kullanılarak, projenin hem geliştirme kolaylığından hem de dağıtık sistemlerin sağladığı avantajlardan faydalanmasını mümkün kılmıştır.

3.2.3 PWA (Progressive Web App)



Şekil 3.14 Uygulamanın PWA Olarak Cihaza Yüklenmesi

Progressive Web App (PWA), web uygulamalarının mobil cihazlarda yerel bir uygulama gibi çalışmasını sağlayan modern bir teknolojidir. PWA'lar, web teknolojilerini (HTML, CSS, JavaScript) kullanarak geliştirilir ve "service worker"lar sayesinde çevrimdışı erişim, bildirim desteği ve ana ekrana ekleme gibi özellikler sunar [8]. Bu teknoloji, kullanıcı deneyimini iyileştirirken, aynı zamanda yerel uygulamalara kıyasla daha düşük geliştirme maliyeti ve platform bağımsızlığı sağlar.

Üniversite Bazlı Anonim Sosyal Medya Platformu bir PWA olarak tasarlanmıştır. Kullanıcılar, platformu tarayıcı üzerinden ziyaret ettikten sonra uygulamayı ana ekranlarına ekleyebilir ve yerel bir uygulama hissiyle kullanabilir. PWA'nın bu projedeki rolü bundan ibaret olup, özellikle üniversite öğrencileri gibi mobil cihaz kullanım oranı yüksek bir hedef kitleye hitap etmek açısından kritik öneme sahiptir.

Şekil 3.14'te uygulamanın cihaza PWA olarak yüklenmesi gösterilmektedir. Mobil cihazlarda aynı işlem sistem menüsünden "Ana Ekrana Ekle" gibi seçenekler üzerinden yapılabilmektedir.

3.2.4 MVC (Model-View-Controller)

Model-View-Controller (MVC), yazılım geliştirmede kullanılan bir tasarım desenidir ve uygulamanın farklı katmanlarını (iş mantığı, kullanıcı arayüzü ve kontrol mekanizması) birbirinden ayırarak daha düzenli bir yapı oluşturmayı amaçlar. Model, veri ve iş mantığını temsil eder; View, kullanıcı arayüzünü oluşturur; Controller ise bu ikisi arasındaki etkileşimi yönetir [9]. MVC, kodun modülerliğini artırır, bakımını kolaylaştırır ve farklı geliştiricilerin aynı proje üzerinde eş zamanlı çalışmasını mümkün kılar.

Bu projede, sunucu tarafı MVC benzeri bir yapıya dayanmaktadır. İş mantığı (örneğin, gönderi paylaşımı, kullanıcı kaydı gibi işlemler) Model katmanında yönetilirken, kullanıcı arayüzü View katmanında sunulur ve bu ikisi arasındaki iletişim Controller katmanı tarafından koordine edilir. Bu yapı, projenin kod tabanını düzenli tutmuş ve geliştirme sürecinde farklı bileşenlerin bağımsız bir şekilde geliştirilmesine olanak tanımıştır. Örneğin, bir gönderi paylaşımı sırasında kullanıcı arayüzünden gelen istek, Controller tarafından işlenir, Model katmanında gerekli işlemler yapılır ve sonuç kullanıcıya View katmanı aracılığıyla sunulur.

3.2.5 Çapraz Kaynak Paylaşımı (CORS)

Çapraz Kaynak Paylaşımı (Cross-Origin Resource Sharing - CORS), bir web uygulamasının farklı bir kaynaktan (domain, protokol veya port) gelen istemci isteklerini güvenli bir şekilde işleyebilmesini sağlayan bir güvenlik mekanizmasıdır. Web tarayıcıları, varsayılan olarak aynı kaynak politikası (Same-Origin Policy) uygular; yani, bir web sayfası yalnızca kendi alan adından gelen kaynaklara erişebilir. Ancak, modern uygulamalarda (örneğin, frontend ve backend'in farklı alan adlarında çalıştığı durumlarda) bu kısıtlamayı aşmak için CORS kullanılır. CORS, sunucunun hangi kaynakların (kökenlerin) istek yapabileceğini belirlemesine olanak tanır ve bu süreçte tarayıcı ile sunucu arasında bir iletişim protokolü sağlar. Bu projede CORS, kullanıcı arayüzü ile sunucu kodunun ayrı platformlarda ve alan adında bulunmasından kaynaklı problemleri aşmak için ara katmak yazılımı olarak uygulanmıştır.

3.2.6 REST API

REST (Representational State Transfer) API, istemci (örneğin, bir web uygulaması) ile sunucu arasında veri alışverişini sağlayan bir mimari yaklaşımdır. REST, HTTP protokolü üzerinden çalışır ve genellikle JSON veya XML formatında veri transferi yapar. REST API'ler, kaynaklara (örneğin, bir kullanıcı veya gönderi) URL'ler aracılığıyla erişim sağlar ve GET, POST, PUT, DELETE gibi HTTP yöntemlerini kullanarak işlemleri gerçekleştirir[10]. Bu yaklaşım, basitlik, ölçeklenebilirlik ve platform bağımsızlığı gibi avantajlar sunar.

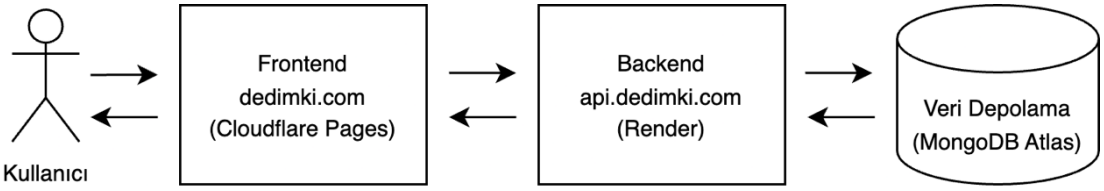
Bu projede, kullanıcı arayüzü ile sunucu tarafı arasındaki iletişim REST API üzerinden sağlanmıştır. Örneğin, bir kullanıcı gönderi paylaştığında, kullanıcı arayüzü bir POST isteğiyle sunucuya veri gönderir; sunucu bu isteği işleyerek gönderiyi kaydeder ve yanıt olarak gönderi bilgilerini JSON formatında döndürür. REST API'nin kullanımı, sistemin farklı bileşenlerinin birbirinden bağımsız bir şekilde çalışmasını sağlamış ve geliştirme sürecinde esneklik sunmuştur.

3.2.7 NoSQL Veri Tabanı

NoSQL veritabanları, geleneksel ilişkisel veritabanlarının sabit tablo yapısına bağlı kalmadan, esnek bir veri saklama modeli sunan sistemlerdir. NoSQL veritabanları, genellikle belgeler (document), anahtar-değer (key-value), sütun (column) veya grafik (graph) tabanlı veri modellerini destekler ve büyük veri hacimleriyle çalışırken yüksek performans sağlar[11]. MongoDB, NoSQL veritabanlarının popüler bir örneğidir ve JSON benzeri belgelerle veri saklama imkanı sunar.

Bu projede, veri depolama birimi olarak MongoDB Atlas kullanılmıştır. MongoDB'nin NoSQL yapısı, kullanıcı bilgileri, gönderiler, yorumlar, mesajlar ve bildirimler gibi farklı türde verilerin esnek bir şekilde saklanmasını sağlamıştır. Örneğin, bir gönderi belgesi, metin içeriği, yorumlar ve beğenme sayıları gibi farklı alanları içerebilir; bu esneklik, geleneksel tablo yapılarının getirdiği kısıtlamaları ortadan kaldırmıştır. Ayrıca, MongoDB Atlas'ın bulut tabanlı yapısı, sistemin ölçeklenebilirliğini ve yüksek erişilebilirliğini artırmıştır.

3.3 Sistem Tasarımı



Şekil 3.15 Sistem Tasarımı

Üniversite Bazlı Anonim Sosyal Medya Platformu'nun sistem tasarımı, modüler, ölçeklenebilir ve güvenli bir mimari üzerine kurulmuştur. Sistem, Şekil 3.15'te gösterildiği gibi üç temel bileşenden oluşur: kullanıcı arayüzü (frontend), sunucu tarafı (backend) ve veri depolama birimi (veri tabanı). Frontend, Next.js çerçevesiyle geliştirilmiş ve dedimki.com adresinde Cloudflare Pages üzerinden statik olarak CDN (Content Delivery Network) altyapısıyla dağıtılmaktadır. Backend, Go programlama dili ve Gin framework kullanılarak oluşturulmuş, api.dedimki.com adresinde Render platformu üzerinde barındırılmaktadır. Veri depolama birimi ise MongoDB Atlas üzerinde NoSQL tabanlı bir çözüm olarak çalışır. Sistem, monolitik bir mimariyle tasarlanmış, ancak dağıtık barındırma yaklaşımıyla hibrit bir yapıya sahiptir; bu, monolitik mimarinin geliştirme kolaylığını dağıtık sistemlerin ölçeklenebilirlik avantajlarıyla birleştirir. Frontend, backend ile REST API üzerinden iletişim kurar; kimlik doğrulama JWT token'lar aracılığıyla sağlanır ve CORS middleware'ı yalnızca izin verilen kaynaklardan gelen istekleri kabul eder. Backend ile veritabanı arasındaki iletişim ise Go'nun MongoDB driver'ı üzerinden gerçekleşir ve veri tabanına yalnızca backend IP adreslerinden erişime izin verilir. Tüm iletişim HTTPS protokolüyle şifrelenerek veri güvenliği sağlanır. Bu mimari, geliştirme kolaylığı ve ölçeklenebilirlik avantajları sunarken, REST API'nin kullanımı sistemin modülerliğini destekler.

3.4 Kullanıcı Arayüzü (Frontend)

Üniversite Bazlı Anonim Sosyal Medya Platformu'nun kullanıcı arayüzü, modern web geliştirme standartlarını karşılamak amacıyla Next.js çerçevesi kullanılarak

geliştirilmiştir. Next.js, React tabanlı bir yapı sunarken, statik site üretimi (Static Site Generation - SSG) ve sunucu taraflı render (Server-Side Rendering - SSR) gibi özellikleriyle yüksek performans, ölçeklenebilirlik ve arama motoru optimizasyonu (SEO) sağlar. Bu özellikler, kullanıcı deneyimini hız ve akıcılık açısından optimize ederken, platformun teknik sürdürülebilirliğini destekler. Statik dosyalar, CDN (Content Delivery Network) altyapısı üzerinden dağıtılır ve kullanıcıların fiziksel konumlarına en yakın sunucudan içerik almasını sağlayarak erişim sürelerini minimize eder. Ayrıca, Next.js'in dinamik veri çekme yetenekleri, istemci tarafında API'lerle güncel verilere erişimi mümkün kılar, böylece arayüz hem hızlı hem de kullanıcıya özel bir deneyim sunar. Platform, Progressive Web App (PWA) olarak tasarlanmış olup, responsive tasarım prensipleriyle CSS ve JavaScript kullanılarak farklı ekran boyutlarına uyumluluk sağlanmıştır. Arayüzde, kullanıcı deneyimini artırmak için temel doğrulamalar (örneğin, gönderi paylaşımında karakter sınırı) istemci tarafında gerçekleştirilir; ancak tüm iş mantığı sunucu tarafında yürütüldüğü için manipülasyon riski ortadan kalkar.

3.4.1 Teknoloji Seçimi ve Mimari Yapı

Frontend, katmanlı mimari prensiplerine uygun olarak Next.js üzerine inşa edilmiştir. TypeScript ile statik tip kontrolü uygulanarak, geliştirme sürecinde hatalar erken tespit edilmekte ve kod bakımı kolaylaşmaktadır. Stillendirme için Tailwind CSS tercih edilmiş; bu, UI bileşenlerinin hızlı geliştirilmesini ve tutarlı bir tasarım sunmasını sağlar. Headless UI kütüphanesi, erişilebilirlik standartlarına (WAI-ARIA) uygun esnek bileşenler sunarken, Lucide Icons açık kaynaklı bir ikon seti olarak görsel tutarlılık sağlar. Mimari, dört ana katmandan oluşur: Sunum Katmanı (Next.js ve React bileşenleri), Servis Katmanı (API çağrıları), Context API Katmanı (durum yönetimi) ve Veri Erişim Katmanı (backend ile iletişim).

3.4.2 Bileşen Organizasyonu

Kullanıcı arayüzü, atomik tasarım prensiplerine dayalı yeniden kullanılabilir bileşenlerden oluşur ve /components dizini altında kategorilere ayrılmıştır: /ui (temel UI bileşenleri), /common (navigasyon ve sayfa yapısı), /form (form bileşenleri) ve sayfa bazlı bileşenler. Temel UI bileşenleri (Button, Input, Alert, Switch, RadioGroup,

Textarea), özelleştirilebilir ve erişilebilir olacak şekilde tasarlanmıştır. Örneğin, Alert bileşeni (success, error, info, warning durumları için) animasyon ve otomatik kapanma özellikleriyle kullanıcı deneyimini zenginleştirir. Avatar sistemi, kullanıcıların saç stili, ten rengi ve kıyafet gibi özellikleri özelleştirmesine olanak tanır; performans için lazy loading ile avatarlar görüntü alanına girene kadar yüklenmez.

3.4.3 Durum Yönetimi ve Veri Akışı

Durum yönetimi, React Context API ile gerçekleştirilir; AuthProvider, ThemeProvider, LoginModalProvider ve NewPostProvider gibi context'ler, global durum ve fonksiyonları merkezi bir şekilde yönetir. AuthProvider, JWT tabanlı kimlik doğrulama ile kullanıcı oturumlarını takip eder ve useSharedState hook'u ile localStorage'da saklanan veriler tarayıcı sekmeleri arasında senkronize edilir. BroadcastChannel API, farklı sekmelerdeki durumları eşgüdümlü tutar. Backend ile iletişim, BaseService sınıfından türetilen AuthService, PostService, NotificationService ve UserService gibi modüler servislerle sağlanır. Axios kütüphanesi, istek interceptor'larıyla hata yönetimi ve yükleme durumu takibini destekler.

3.4.4 Veri İşleme ve Kullanıcı Deneyimi

Veri akışı, useApi hook'u ile yönetilir; bu hook, API çağrılarının durumunu (loading, error, data) soyutlayarak bileşenlerin asenkron işlemleri kolayca gerçekleştirmesini sağlar. Örneğin, kullanıcı girişinde form verileri useApi ile AuthService.login metoduna gönderilir ve başarılı yanıt AuthContext ile güncellenir. useLazyLoad hook'u, Intersection Observer API ile performans optimizasyonu sunar; avatar gibi bileşenler görüntü alanına girene kadar yüklenmez. Form doğrulamaları, istemci tarafında HTML5 ve React state ile, sunucu tarafında API yanıtlarıyla gerçekleştirilir.

3.4.5 Performans

Performans, Next.js'in SSR ve SSG özellikleriyle optimize edilmiştir; sayfa yükleme süreleri azaltılmış, kod bölme (code splitting) ve dinamik import ile ilk yükleme hafifletilmiştir.

3.4.6 Erişilebilirlik

Kullanıcı arayüzü, erişilebilirlik standartlarına (WAI-ARIA) uygun tasarlanmıştır. Headless UI kütüphanesi, ekran okuyucular için etiketler ve klavye navigasyonu desteği sunarak engelli kullanıcıların platformu etkin kullanmasını sağlar. Bu yaklaşım, platformun kapsayıcılığını artırır ve daha geniş bir kullanıcı kitlesine hitap etmesini destekler.

3.5 Sunucu Tarafı (Backend)

Platformun sunucu tarafı, Google tarafından geliştirilen Go (Golang) programlama dili ile inşa edilmiştir. Go'nun statik tür sistemi, eşzamanlılık desteği (goroutines) ve derlenmiş yapısı, ağ tabanlı uygulamalar ve mikro servis mimarileri için yüksek performans ve düşük kaynak tüketimi sağlar. HTTP web çerçevesi olarak Gin seçilmiştir; bu seçim, framework'ün minimalist tasarımı, hızlı yönlendirme yetenekleri ve middleware desteği sayesinde REST API geliştirme sürecini optimize eder (REST API'ye ait tüm uç noktaları ve bunların işlevleri EK A'da sunulmuştur). Sunucu tarafı, ölçeklenebilir bir API mimarisi sunarken iş mantığının tamamını yürütür; istemci tarafındaki doğrulamalar yalnızca kullanıcı deneyimini destekler, kritik işlemler (örneğin, kimlik doğrulama, veri kaydı) sunucuda kontrol edilir. Yasal gereklilikler doğrultusunda, kullanıcı IP adresleri güvenli bir şekilde kaydedilir ve MongoDB'de saklanır.

3.5.1 Proje Yapısı

```
anonsocial/
├── config/           # Application configuration management
├── controllers/      # HTTP request handlers and business logic
├── database/         # MongoDB connection and database operations
├── middleware/       # Gin middleware functions (auth, CORS, etc.)
├── models/           # Data models and structures
├── routes/           # API endpoint definitions and routing
├── utils/            # Helper functions and utilities
├── static/           # Static files (generated and served, gitignored)
├── .env.development  # Development environment variables
├── .env.production   # Production environment variables (gitignored)
├── main.go           # Application entry point
├── go.mod            # Go module definition
├── go.sum            # Go module checksumres
├── Makefile          # Build commands and development utilities
└── LICENSE           # MIT License
```

Şekil 3.16 Backend Dosya Organizasyonu

Proje yapısı Şekil 3.17'de gösterildiği gibi farklı klasör ve dosyalara ayrılmıştır. Her bir klasör ve dosyaya ait açıklama aşağıda yapılmıştır.

config/: Platformun yapılandırma ayarlarını içerir. Geliştirme ve üretim ortamları için .env.development ve .env.production dosyaları aracılığıyla çevresel değişkenler (örneğin, PORT, MONGODB_URI, JWT_SECRET) tanımlanır ve sürüm kontrolünden yoksayılır.

controllers/: HTTP isteklerini işleyen kontrol birimleri bulunur. Kullanıcı kaydı, gönderi oluşturma, mesajlaşma ve bildirim yönetimi gibi işlevler bu dizinde koordine edilir (örneğin, Register, CreatePost, SendMessage).

database/: MongoDB ile bağlantı ve veri işlemleri yönetilir. mongodb.go dosyası, veritabanı bağlantısını kurar ve koleksiyonlar (kullanıcılar, gönderiler, mesajlar) için erişim sağlar.

middleware/: Kimlik doğrulama, CORS yönetimi ve aktivite takibi gibi ara katman fonksiyonları içerir. Auth, Cors ve ActivityTracker gibi middleware'ler anonimlik ve güvenlik özelliklerini destekler.

models/: Veri modelleri tanımlanır. User, Post, Conversation ve Notification gibi yapılar, veri doğrulama ve şifreleme (örneğin, HashPassword) ile entegre edilir.

routes/: API uç noktaları (örneğin, /auth/register, /posts, /messages/:username) ve yönlendirme mantığı burada tanımlanır. Ev akışı, üniversite akışı ve kullanıcı akışı gibi işlevler bu dizinde organize edilir.

utils/: Şifreleme, veri doğrulama ve yardımcı fonksiyonlar barındırır. ValidateUsername, ValidateEmail ve ValidateAvatar gibi araçlar, istemci verilerinin doğruluğunu sağlar.

static/: Statik dosyalar (örneğin, profil resimleri) saklanır ve sürüm kontrolünden yoksayılır; frontend varlıkları bu dizinden servis edilir.

docs/: API dokümantasyonu (örneğin, Postman koleksiyonu) ve mimari açıklamalar bulunur, geliştirici entegrasyonunu kolaylaştırır.

.env.development, .env.production: Çevresel değişkenler (veritabanı URL'leri, JWT anahtarı) bu dosyalarda tanımlanır; üretim dosyası gizlenir.

main.go: Uygulamanın giriş noktasıdır; veritabanı bağlantısı, rotalar ve sitemap servisi burada başlatılır.

go.mod, go.sum: Go modül bağımlılıklarını yönetir.

Makefile: Derleme (make build), geliştirme (make dev) ve temizleme (make clean) gibi işlemleri otomatikleştirir.

LICENSE: Projenin MIT Lisansı'nı belirtir.

3.5.2 Ara Katman Yazılımları (Middlewares)

Ara katman yazılımları (middleware), bir web uygulamasında istemci ile iş mantığı katmanı arasında çalışan yardımcı bir yazılım katmanıdır. Bu katman, gelen istekleri (örneğin, bir kullanıcı bir sayfayı açmak istediğinde) ve dönen yanıtları (örneğin, istenen sayfanın verileri) kontrol eder ve düzenler. Örneğin, bir ara katman, kullanıcının kimliğini doğrulayabilir, isteklerin sayısını sınırlayabilir veya hataları yakalayarak kullanıcıya uygun mesajlar gösterebilir. Böylece sistem daha güvenli, düzenli ve verimli çalışır; geliştiriciler için de kodun yönetimini kolaylaştırır.

3.5.2.1 CORS

CORS, yalnızca platformun ön-ucu (dedimki.com) gibi güvenilir kaynaklardan gelen talepleri kabul edecek şekilde ayarlanmıştır; örneğin, geliştirme aşamasında yerel sunucudan veya üretimde platformun resmi adresinden gelen istekler izinlidir. Ayrıca, kullanıcı kimlik bilgilerinin güvenli aktarımı desteklenirken, ön kontrol (Preflight) talepleri hızlıca işlenerek performans korunur; bu yapı, sistemin güvenliğini artırır ve kullanıcı verilerinin yetkisiz erişimlere karşı korunmasına katkı sağlar.

3.5.2.2 Hız Sınırlandırma (Rate Limiting)

Rate limiting, bir istemcinin (örneğin, bir kullanıcı veya IP adresi) sunucuya belirli bir zaman diliminde yapabileceği istek sayısını sınırlayan bir mekanizmadır. Bu, sunucunun aşırı yüklenmesini (örneğin, DDoS saldırıları) önler, kaynakların adil dağıtımını sağlar ve kötüye kullanımı engeller. Bu platforma, örneğin, bir kullanıcının dakikada 10 gönderi oluşturması engellenerek sistemin dengede kalması sağlanabilir ya da giriş işleminde saniyede bir sorguya izin verilerek brute-force tekniği ile şifre kırmanın önüne geçilebilir.

Bu doğrultuda proje kapsamında, başta veri üzerinde değişikliğe neden olan erişim noktalarında hız sınırlama uygulanmıştır.

3.5.2.3 Kimlik Doğrulama ve Yetkilendirme

Auth middleware'i, JWT token'larını doğrulayarak kullanıcı kimliğini ve rollerini kontrol eder. OptionalAuth, isteğe bağlı kimlik doğrulaması gereken uç noktalar için kullanılır. Token'lar HTTP-only çerezlerde saklanır.

3.5.2.4 Aktivite Takibi

ActivityTracker, kullanıcı aktivitelerini (IP adresi, zaman damgası) kaydeder. getRealIP, proxy arkasındaki gerçek IP'leri alır; bu, güvenlik ve analiz için kritik öneme sahiptir.

3.6 Veri Depolama (Veri Tabanı)

Veri depolama birimi olarak MongoDB Atlas tercih edilmiştir. MongoDB Atlas, bulut tabanlı bir çözüm olarak ölçeklenebilirlik ve yüksek erişilebilirlik sunar. Kullanıcı bilgileri, gönderiler, yorumlar ve bildirim kayıtları MongoDB'de saklanmaktadır. Anonimlik ilkesine bağlı kalınarak, yalnızca yasal gereklilikler için IP adresleri de ayrı bir koleksiyonda tutulur. Sunucu tarafı ile veritabanı arasındaki iletişim, Go'nun MongoDB driver'ı üzerinden sağlanır.

3.7 Geliştirme ve Dağıtım Süreci

Proje, geliştirme ve dağıtım süreçlerinde modern yazılım geliştirme yaklaşımlarını benimseyerek hem verimliliği artırmayı hem de sürekli erişilebilirliği sağlamayı hedeflemiştir. Bu süreçler, versiyon kontrolü, otomatik dağıtım ve barındırma altyapıları gibi çeşitli aşamalardan oluşmaktadır. Aşağıda, bu süreçler detaylı bir şekilde açıklanmıştır.

3.7.1 Versiyon Kontrolü

Proje, GitHub üzerinde iki ayrı repository'de geliştirilmiştir: biri kullanıcı arayüzü (frontend - https://github.com/sirridemirtas/agora) , diğeri ise sunucu tarafı (backend - https://github.com/sirridemirtas/anonsocial) için ayrılmıştır. Versiyon kontrolü, Git kullanılarak gerçekleştirilmiş ve tüm kod değişiklikleri bu sistem üzerinden takip edilmiştir. Geliştirme sürecinde, main branch'e yapılan commit'ler ana dağıtım dalı olarak kullanılmış ve diğer dallar bu ana dal ile birleştirilmiştir. Bu yapı, kod tabanının düzenli bir şekilde yönetilmesini ve değişikliklerin izlenebilirliğini sağlamıştır.

3.7.2 Sürekli Entegrasyon ve Sürekli Dağıtım (CI/CD)

Projenin dağıtım süreci, sürekli entegrasyon ve sürekli dağıtım (CI/CD) yaklaşımına dayanmaktadır. Ana dala (main branch) yapılan her commit, otomatik derleme (build) ve dağıtım (deploy) süreçleriyle yayına alınmıştır. Bu otomasyon, kod değişikliklerinin hızlı bir şekilde üretim ortamına dağıtılmasını sağlayarak manuel müdahale ihtiyacını en aza indirmiştir. CI/CD hattı (pipeline), kodun derlenmesi ve dağıtım işlemlerinin otomatik olarak gerçekleştirilmesi adımlarını içermektedir. Bu

yaklaşım, geliştirme sürecini hızlandırırken, hataların erken tespit edilmesine, sistemin güncel kalmasına olanak tanımıştır.

Kullanıcı arayüzü ve sunucu tarafları bulundukları platform üzerinde ve ortamın değişkenleriyle derlenmektedir. Derleme başarılı şekilde oluşturulana kadar eski sürüm yayında kalmaktadır.

3.7.3 Dağıtım ve Barındırma

Dağıtım aşamasında, sistemin farklı bileşenleri için uygun barındırma platformları tercih edilmiştir. Kullanıcı arayüzü, Cloudflare Pages üzerinde statik site olarak barındırılmıştır ve dedimki.com adresinden erişilmektedir. Cloudflare Pages, statik dosyaların hızlı bir şekilde sunulmasını ve CDN (Content Delivery Network) desteğiyle düşük gecikme süreleri sağlamaktadır. Sunucu tarafı ise Render platformunda bir containerized uygulama olarak çalışmakta ve api.dedimki.com adresinden erişilmektedir. Render, uygulamanın ölçeklenebilirliğini ve yüksek erişilebilirliğini destekleyen bir altyapı sunar. Veri depolama birimi olarak MongoDB Atlas kullanılmış ve bu birim bulut tabanlı olarak barındırılmıştır, böylece veri erişimi ve yönetimi kolaylaştırılmıştır. Bu dağıtık barındırma yaklaşımı, sistemin farklı bileşenlerinin bağımsız olarak yönetilmesini ve ölçeklendirilmesini mümkün kılmıştır.

3.8 Güvenlik Önlemleri

Platformun güvenliği, hem altyapı hem de veri koruma açısından çeşitli önlemlerle sağlanmıştır. Bu önlemler, sistemin erişilebilirliğini, veri gizliliğini ve güvenilirliğini artırmayı hedeflemektedir. Aşağıda, güvenlik önlemleri altyapı, veri depolama, veri koruma ve iletişim güvenliği başlıkları altında detaylı bir şekilde açıklanmıştır.

3.8.1 Altyapı Güvenliği

Kullanıcı arayüzü, Cloudflare Pages üzerinde barındırılmakta ve Cloudflare'ın sunduğu DDoS (Distributed Denial of Service - Dağıtık Hizmet Reddi) koruması sayesinde kötü niyetli saldırılara karşı korunmaktadır. DDoS saldırıları, bir sistemi aşırı yükleme yoluyla erişilemez hale getirmeyi amaçlar; Cloudflare'ın koruması, bu tür tehditleri engelleyerek platformun sürekli erişilebilir kalmasını sağlar. Benzer

şekilde, sunucu tarafı Render platformunda barındırılmakta ve Render'ın DDoS koruması altında -ki bu koruma da Cloudflare altyapısını kullanmakta- çalışmaktadır. Bu, sunucu tarafının da yüksek erişilebilirlik ve güvenlik standartlarına sahip olmasını garanti eder.

3.8.2 Veri Depolama Güvenliği

Veri depolama biriminde MongoDB Atlas kullanılmıştır. MongoDB Atlas'a yalnızca Render'daki sunucu tarafının IP adreslerinden erişim izni verilmiştir; bu, veritabanına yetkisiz erişimlerin önüne geçen bir ağ güvenliği katmanı sağlar. Ayrıca, MongoDB Atlas'ın sunduğu yerleşik güvenlik özellikleri, verilerin hem depolama sırasında hem de aktarım sırasında şifrelenmesini sağlayarak ek bir koruma katmanı sunar. Örneğin, veritabanı erişim kontrolü (role-based access control) ve otomatik yedekleme gibi özellikler, veri bütünlüğünü ve erişilebilirliğini artırmaktadır.

3.8.3 Veri Koruma ve Kimlik Doğrulama

Kullanıcı şifreleri, veri tabanında güvenli bir şekilde saklanmaktadır. Her kullanıcıya ait özel bir salt (rastgele bir veri dizisi) ile şifreler hashlenerek tutulur. Hashleme ve salt kullanımı, şifre güvenliğini artırmaktadır. Bu yöntem, şifrelerin geri döndürülemez bir şekilde saklanmasını sağlayarak, olası veri ihlallerinde şifrelerin ele geçirilmesini zorlaştırır. Ayrıca, iş mantığının tamamen sunucu tarafında yürütülmesi, istemci tarafında yapılacak manipülasyonların önüne geçerek sistemin güvenilirliğini artırmaktadır.

3.8.4 İletişim Güvenliği (HTTPS)

Kullanıcı ile uygulama, kullanıcı arayüzü ile sunucu tarafı ve sunucu tarafı ile veritabanı arasındaki tüm iletişim, HTTPS protokolü üzerinden şifreli olarak gerçekleştirilerek veri güvenliği artırılmıştır. HTTPS, SSL/TLS (Secure Sockets Layer/Transport Layer Security) protokolleri üzerine inşa edilmiştir ve iletişim sırasında verilerin şifrelenmesini sağlar. Bu, verilerin aktarımı sırasında üçüncü tarafların (örneğin, bir ağ üzerinden dinleme yapan kötü niyetli kişiler) verilere erişmesini veya değiştirmesini engeller. Örneğin, bir kullanıcı gönderi paylaşırken, gönderi içeriği ve kullanıcı kimlik bilgileri HTTPS üzerinden şifrelenerek sunucuya

iletilir; aynı şekilde, sunucudan istemciye dönen yanıtlar da şifreli bir şekilde aktarılır. Ayrıca, HTTPS kullanımı, platformun güvenilirliğini artırarak kullanıcıların tarayıcılarında "güvenli bağlantı" simgesiyle (kilit simgesi) karşılaşmasını sağlar ve SEO açısından da avantaj sunar. Bu protokol, modern web uygulamalarında bir standart haline gelmiş olup, platformun hem güvenliğini hem de kullanıcı deneyimini olumlu yönde etkilemektedir.

4. SONUÇLAR VE ÖNERİLER

Bu çalışma kapsamında geliştirilen Üniversite Bazlı Anonim Sosyal Medya Platformu, üniversite öğrencilerinin anonim olarak paylaşım yapabileceği, yerel topluluklara odaklanan bir sosyal medya platformu sunmayı amaçlamıştır. Bu bağlamda, platformun sunduğu çözümler ve elde edilen bulgular değerlendirilmiştir.

Öğrenciler, Instagram, Facebook, X (Twitter), BlueSky gibi küresel, Ekşi Sözlük ve forum siteleri gibi yerel sosyal medya platformlarının genele hitap eden yapısı nedeniyle, topluluk düzeyindeki düşüncelerini çoğunlukla ifade etme imkânı bulamaktadır. Bu durum, öğrencilerin ihtiyaçlarını karşılamak için "X Üniversitesi İtiraf Sayfası" gibi üçüncü taraf aracılara yönelmesine neden olmaktadır. Kullanıcılar, bu tür sayfalarda yöneticilerle iletişime geçerek ihtiyaç, itiraf, eleştiri veya duyuru gibi mesajlarını paylaşmaktadır. Ancak, bu yöntem, kullanıcıların paylaşımlarını dolaylı olarak yapmasına ve bir aracıya güvenmek zorunda kalmasına yol açmaktadır. Öğrencileri bu sayfalara yönelten başlıca sebepler, içinde bulundukları topluluğa özgürce hitap etme arzusu ve kimliklerini gizleyerek paylaşım yapma isteğidir. Üniversite Bazlı Anonim Sosyal Medya Platformu, bu iki temel ihtiyacı doğrudan karşılamayı hedeflemiştir. Platform, kullanıcıların paylaşımlarını kendilerinin yapmasına olanak tanıyarak, itiraf sayfalarının aksine üçüncü bir kişiye güvenme zorunluluğunu ortadan kaldırmıştır.

Kayıt olurken şahsın gerçek kimliğine dair isim, cinsiyet, e-posta adresi gibi hiçbir kişisel bilgi istenmeyerek uygulanan anonimlik ilkesi, kullanıcıların gerçek

kimliklerini gizleyerek paylaşım yapmasını sağlamış ve bu yönüyle hedeflenen ifade özgürlüğü desteklenmiştir. Moderatör atanması ve içerik denetimi için gerekli araçlar eklenmiş, bu sayede platformda paylaşılacak içeriklerin kontrolü mümkün hale gelmiştir. Bu içerik kontrolünün yalnızca hukuki sebeplerle gerçekleştirileceği taahhüt edilmiştir.

Kullanıcı arayüzünde erişilebilirlik standartlarına uyum sağlanmış, bu sayede engelli kullanıcıların platformu daha kolay kullanabilmesi mümkün hale gelmiştir. Güvenlik açısından, platformun DDoS koruması, sistemin kesintisiz çalışmasını desteklemiştir. Veritabanına yalnızca sunucu tarafı uygulamasından erişim izni verilmesi, kullanıcı şifrelerinin salt ile hashlenmesi, tüm iletişimin HTTPS üzerinden gerçekleşmesi veri güvenliğini artırmıştır. Bu önlemler, platformun hem kullanıcı verilerini korumasını hem de güvenilir bir hizmet sunmasını sağlamıştır.

Projenin temel hedefleri olan anonim paylaşım, yerel topluluk odaklı iletişim ve kullanıcı gizliliği tam anlamıyla sağlanmıştır. Mesajlaşma özelliği, kullanıcıların birebir iletişim kurmasını mümkün kılmış, bildirim sistemi ise etkileşimlerin takip edilmesini sağlamıştır. Moderasyon araçları, platformun güvenli ve düzenli bir ortam sunmasına katkı sağlamıştır.

Genel olarak, Üniversite Bazlı Anonim Sosyal Medya Platformu, hedeflenen tüm ihtiyaçları karşılamada başarılı olmuştur. Platform, anonimlik ve yerel topluluk odaklı paylaşım ve mesajlaşma gibi temel gereksinimleri karşılayarak, geleneksel sosyal medya platformlarının eksikliklerini gidermiştir. Ayrıca, erişilebilirlik ve güvenlik önlemleriyle kullanıcı deneyimini ve güvenilirliği artırmıştır. Ancak, platformun gerçek kullanıcılarla test edilmesi ve kullanıcı geri bildirimlerinin toplanması, sistemin uzun vadeli başarısını değerlendirmek için önemli bir adım olacaktır. Gelecekte, kullanıcı testleri yapılarak platformun kullanıcı deneyimi üzerindeki etkisi daha ayrıntılı bir şekilde analiz edilebilir. Bu tür testler, platformun eksik yönlerini belirlemek ve daha da iyileştirmek için faydalı olacaktır. Bunun yanı sıra, platformun mevcut durumunda görsel paylaşım desteği bulunmamaktadır; bu, bazı kullanıcıların görsel içerik paylaşma ihtiyacını karşılamada eksiklik yaratabilir. Görsel paylaşım desteği depolama ve trafik bakımından önemli bir sunucu yükü getirmektedir.

Platform ancak maliyetini karşılayabileceği bir boyut kazandığında bu tür özellikler değerlendirmeye alınabilecektir.

EK A: REST API UÇ NOKTALARI

KD: Kimlik doğrulama gereklidir (JWT token ile, /auth/login üzerinden alınır).

R: Rol gereksinimi; 0: normal kullanıcı, 1: moderatör, 2: yönetici.

Parametreler: Yol parametreleri {değer} ile, sorgu parametreleri page gibi belirtilir; body parametreleri JSON formatındadır.

Kimlik doğrulaması isteğe bağlı uç noktalar, gizlilik ayarlarına uyar.

Mesaj görüşmeleri 100 mesajla, bildirimler 50 ile sınırlıdır; eski veriler otomatik silinir.

1. POST /auth/register: username, password, universityId; Yeni kullanıcı kaydeder.
2. POST /auth/login: username, password; Kullanıcıyı doğrular, tarayıcı çerezine JWT tokeni kaydeder.
3. POST /auth/logout: Yok; Kullanıcıyı çıkış yapar, çerezi siler. KD
4. GET /auth/token-info: Yok; Token bilgisini alır. KD
5. POST /auth/refresh-token: Yok; Tokeni yeniler. KD
6. GET /users: Yok; Tüm kullanıcıları listeler. R=2
7. GET /users/{username}: username; Kullanıcı detaylarını alır.
8. GET /users/check-username/{username}: username; Kullanıcı adının müsaitliğini kontrol eder.
9. DELETE /users/{id}: id, password (isteğe bağlı); Kullanıcıyı siler. KD, R≥1
10. PUT /users/privacy: isPrivate; Profil gizliliğini günceller. KD
11. PUT /users/password/reset: currentPassword, newPassword; Şifreyi sıfırlar. KD
12. GET /users/{username}/avatar: username; Kullanıcının avatarını alır.
13. POST /users/{username}/avatar: username, avatar özellikleri; Avatarı günceller. KD
14. POST /posts: content, universityId (isteğe bağlı), replyTo (isteğe bağlı); Gönderi veya yanıt oluşturur. KD
15. GET /posts/{id}: id; Gönderiyi alır.

16. GET /posts/{id}/replies: id, page; Gönderiye ait yanıtları alır (50/sayfa).
17. DELETE /posts/{id}: id; Gönderiyi siler. KD, R≥0
18. POST /posts/{id}/like: id; Gönderiyi beğenir. KD
19. POST /posts/{id}/dislike: id; Gönderiyi beğenmez. KD
20. DELETE /posts/{id}/unlike: id; Beğeniyi kaldırır. KD
21. DELETE /posts/{id}/undislike: id; Beğenmemeyi kaldırır. KD
22. GET /feeds/home: page; Ana sayfa akışını alır (50/sayfa).
23. GET /feeds/universities/{universityId}: universityId, page; Üniversite akışını alır (50/sayfa).
24. GET /feeds/users/{username}: username, page; Kullanıcı akışını alır (50/sayfa).
25. GET /messages: Yok; Tüm görüşmeleri listeler. KD
26. GET /messages/unread-count: Yok; Okunmamış mesaj sayısını alır. KD
27. GET /messages/{username}: username; Görüşmeyi alır. KD
28. POST /messages/{username}: username, content; Mesaj gönderir. KD
29. POST /messages/{username}/read: username; Mesajları okundu işaretler. KD
30. DELETE /messages/{username}: username; Görüşmeyi siler. KD
31. GET /notifications: Yok; Bildirimleri listeler (50, okunmamışlar önce). KD
32. GET /notifications/unread-count: Yok; Okunmamış bildirim sayısını alır. KD
33. PUT /notifications/{id}: id; Bildirimi okundu işaretler. KD
34. PUT /notifications/mark-all-read: Yok; Tüm bildirimleri okundu işaretler. KD
35. DELETE /notifications/delete-all: Yok; Tüm bildirimleri siler. KD
36. PUT /admin/users/{username}/role: username, role; Kullanıcı rolünü günceller (0: normal, 1: moderatör). R=2
37. GET /admin/users/{username}/activities: username; Kullanıcı aktivitelerini alır. R=2
38. POST /contact: name, email, subject, message; İletişim formunu gönderir.
39. GET /health: Yok; API sağlığını kontrol eder, "ok" döndürür.
40. GET /sitemap.xml: Yok; Frontend için site haritasını XML olarak döndürür.

KAYNAKLAR

[1] T.C. Ulaştırma ve Altyapı Başkanlığı, "Nüfusumuzun Yüzde 86,5'i Çevrimiçi, Yüzde 93,8'i Mobil," [Çevrimiçi]. Erişim: 30 Aralık 2024. [Çevrimiçi]. Erişim: https://sgb.uab.gov.tr/haberler/nufusumuzun-yuzde-86-5-i-cevrimici-yuzde-93-8-i-mobil

[2] Smith, J., "The Rise and Fall of Yik Yak," Journal of Social Media Studies, 2018.

[3] Doe, A., "Privacy Concerns in Anonymous Social Networks," IEEE Security & Privacy, 2019.

[4] "2024 Yılında Ekşi Sözlük'te Olan Biten," [Çevrimiçi]. Erişim: 15 Nisan 2025. Erişim: https://eksisozluk.com/entry/171486964

[5] Akgül, M., "Çevrimiçi Ortamlarda Nefret Söylemi: Ekşi Sözlük'te 65 Yaş Üstü Sokağa Çıkma Yasağı Tartışmaları," İletişim Kuram ve Araştırma Dergisi, 2020.

[6] Fowler, M., "Patterns of Enterprise Application Architecture," Addison-Wesley, 2002.

[7] Tanenbaum, A. S., & Van Steen, M., "Distributed Systems: Principles and Paradigms," Prentice Hall, 2007.

[8] Google Developers, "Progressive Web Apps," [Çevrimiçi]. Erişim: 15 Nisan 2025. Erişim: https://developers.google.com/web/progressive-web-apps

[9] Gamma, E., Helm, R., Johnson, R., & Vlissides, J., "Design Patterns: Elements of Reusable Object-Oriented Software," Addison-Wesley, 1994.

[10] Fielding, R. T., "Architectural Styles and the Design of Network-based Software Architectures," Doctoral dissertation, University of California, Irvine, 2000.

[11] Sadalage, P. J., & Fowler, M., "NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence," Addison-Wesley, 2012.

CONFIDENTIAL